Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#05)

Use of Venus Simulator for Register Level Execution of RISC-V Control Instructions

Student Name: Muhammad Hamza Roll No: 24i-6513 Section: B

Date performed: 09 , Sep , 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

// Write description of the task here 👇 (max. five to seven lines)

This program serves as an introductory computer architecture exercise demonstrating fundamental RISC-V control flow: it illustrates how processors execute conditional branching (`if-else` decision making) and perform basic arithmetic before exiting cleanly.

Code:

```
.data
.text
.globl main
main:
    # This instruction is I type instruction we used save type register s0(x8)
    addi s0, s0, 4
     # This instruction is I type instruction we used save type register s0(x9)
    addi s1, s1, 6
    # it will jump to the section or branch to the Fin if (s0 == s1) other wise it will not jump and run the bellow code
    beq s0, s1, Fin

True:
    add s2, s1, s0
    j end
Fin:
    sub s2, s1, s0

end:
    li a0 , 0
```

Output:

Task 2:

Description:

// Write description of the task here 👇 (max. five to seven lines)

Beq is a conditional branch and it jumps only when the two registers are equal. j is an unconditional jump: and it always jumps to the specified target.

Code:

// Paste source code here 👇

```
.data
.text
.globl main
main:
   #this instruction will basicaly intialize the s0 register with zero value
   addi s0,      s0,      4
   # it will also do addition
         addi    s1,      s1,      4
   # it will jump to the 'Fin' Label if the s0 and s1 register with same values other wise there will
be no jump
         beq     s0,      s1,      Fin
True:
         add     s2,      s1,      s0
         j       End
Fin:
         sub     s2,      s1,      s0
End:
   li a0 , 0
```

Output:

// Import output here 👇

Task 3:

Description:

// Write description of the task here 👇 (max. five to seven lines)

This program illustrates the direct translation of an explicit `if-else` control structure into assembly language: it tests an equality condition with `beq`, demonstrates a fallback path using an unconditional jump (`j else`) when the branch is not taken, and performs the corresponding arithmetic operation before converging at a shared exit label.

Code:

```
.data
.text
.globl main
main:

   addi s0,s0,     5          # a
         addi s1,s1,        10  # b
   addi s2,s2, 4  # d
         addi s3,s3,        3   # e
   beq s0 ,s1 , IF
         j else

IF:
   sub s4, s2 , s3
   j End
else:
         add s4 ,s2 ,s3
   j End
End:
   li a0 , 0
```

Output:

// Import output here 👇

Task 4:

Description:
// Write description of the task here 👇 (max. five to seven lines) This code demonstrates how high-level conditional logic (like an `if (a == b)` block) translates into low-level assembly branching and stack memory management in computer architecture.
Code:

```
.data
.text
.globl
main
main:
    addi s0,s0,5 #s0=a
    addi s1,s1,10 #s1=b
    addi s2,s2,4 #s2=d
    addi s3,s3,3 #s3=e
    beq s0,s1,Fin
    True:
        add s4,s2,s3 #s4=c
        c=d+e j end
    Fin:
        sub s4,s2,s3 #s4=c  c=d-e
end:
sw s4,0(sp)
```

Output:
// Import output here 👇



Task 5:

Description:
// Write description of the task here 👇 (max. five to seven lines) In this code, I stored a random numerical value in s0, and added shift left logical immediate twice by 2 as it was mentioned in the task.
Code:
.data .text .globl main main: li s0,5 slli s1,s0,2 slli s2,s1,2

Output:

// Import output here 👇



Task 6:

Description:

// Write description of the task here 👇 (max. five to seven lines)

In this code, bgt condition is used that will check either I is greater than 5 or not. And for loop I am used jump in Fin that will jump the code to bgt after incrementing counters and index and once the I will become greater than 5 it will go into true and the true will end the program.

Code:

```
.data
.text
.globl main main:
    li t0,0          # t0: index = 0     li t1,5          #
t1: total number of indexes = 5     li t2,0          #
```

```
t2: counter = 0     bgt t1,t0,Fin True:     j end
Fin:
    addi t2,t2,1     # counter++
addi t0,t0,1     # index++
bgt t1,t0,Fin  end:
   sw t2,0(sp)
```

Output:

// Import output here 👇



Task 7:

Description:

// Write description of the task here 👇 (max. five to seven lines)

This RISC-V program initializes an array num and fills its first 5 elements with the values **5, 6, 7, 8, and 9**. The loop uses the index in t0 to calculate each element's memory address and stores the value in t3.

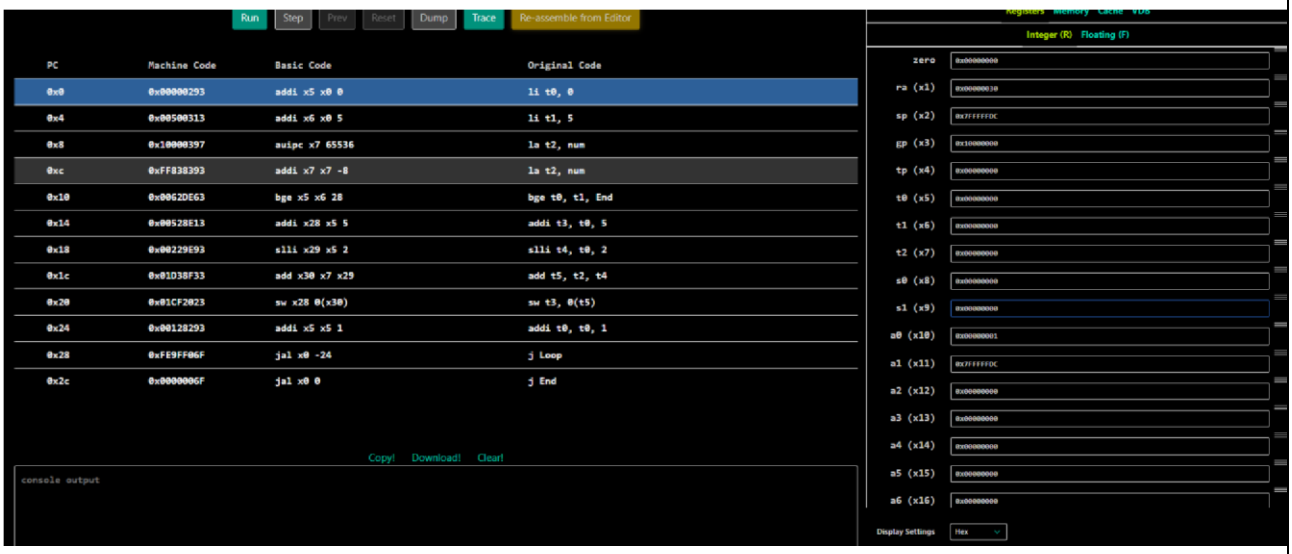
After processing 5 elements, the program reaches End and stays there in an infinite loop.

Code:

```
.data num: .word 0, 0, 0, 0, 0, 0, 0,
0, 0, 0
.text
.globl main main:
    li t0, 0
li t1, 5
la t2, num
Loop:
bge t0, t1, End
addi t3, t0, 5
slli t4, t0, 2
add t5, t2, t4
sw t3, 0(t5)
addi t0, t0, 1
j Loop
End:
    j End
```

Output:

// Import output here



Task 8:

Description:

// Write description of the task here 👇 (max. five to seven lines)

The program initializes x, y, and counter to zero and uses two bgt statements to implement nested loops for x and y. For each combination of x and y, it calculates counter = x + y, then increments y and resets it when moving to the next x. After completing both loops, the final value of counter is stored in memory using sw.

Code:

```
.data
.text
.globl main
main:
    li t0,0 # t0: x-index = 0
    li t1,3 # t1: total number of x indexes =
    3 li t2,0 # t2: counter = 0
    li t3,0 # t3: y-index = 0
    li t4,2 # t4: total number of y indexes =
    2 bgt t1,t0,Fin
    True:
       j
    end
    Fin:
        bgt t4,t3,New
        New_True:
       addi t0,t0,1 # x++ li t3,0 # y = 0
           bgt
           t1,t0,Fin j
           end
       New:
           add t2,t0,t3 # Counter = x + y
           addi t3,t3,1 # y++
           j Fin
       end:
sw t2,0(sp)
```

Output:

// Import output here 👇

